

PA4 实验报告

231180166 杜明阳

实验进度

大部分任务已经完成。

1. 分时多任务的具体过程

目标：让 PAL(仙剑奇侠传) 与 hello 内核线程在 Nanos-lite/AM/NEMU 上交替运行。

- 上下文(Context)的创建与表示
 - 内核线程：在 PCB 的内核栈上用 `kcontext()` 构造，设置 `mepc=entry`，`mstatus.MPP=11b`，并把第一个参数放到 `a0(x10)`（见 `abstract-machine/am/src/riscv/nemu/cte.c`）。
 - 用户进程：在 PCB 的内核栈上用 `ucontext()` 构造，设置 `mepc=entry`，`mstatus.MPP=11b`，`pdir`(若启用 VME)；由 `context_upload()` 分配一片新的用户栈(例如 `new_page(8) → 32KB`)，在其顶部按约定布置 `[argc][argv...][NULL][envp...][NULL]` 和字符串区，将“`argc`的地址”放入 `GPRx(a0)`。用户态 `_start` 把 `sp←a0` 并 `jal call_main`，`crt0.c` 再解析出 `argc/argv/envp`。
- 陷入与保存现场
 - 线程/进程执行 `yield()` 或执行设备访问前的 `yield() → ecall`。
 - 硬件跳到 `mtvec=__am_asm_trap` (`abstract-machine/am/src/riscv/nemu/trap.S`)，将 GPR 全部 PUSH 到当前栈，读取 `mcause/mstatus/mepc` 存入 Context。
 - 调用 `__am_irq_handle(Context *c)` (`cte.c`)，根据寄存器判断：
 - `ECALL` 且 `a7/-1 → EVENT_YIELD`
 - 定时器中断 → `EVENT_IRQ_TIMER` (native 下可能出现)
 - 系统调用 → `EVENT_SYSCALL`
 - 对 `YIELD/SYSCALL`：`c->mepc += 4` 跳过 `ecall`。
- 调度点与选择新上下文
 - Nanos-lite 的 `irq.c/do_event()` 收到 `EVENT_YIELD` 调用 `schedule(c)`。
 - `schedule()` 将当前 PCB 的 `cp` 保存为 `prev`，选择下一个 PCB（示例为双 PCB 轮转，或以 `idle` 兜底），返回目标 PCB 的 `cp`。
- 恢复与切换
 - `__am_irq_handle` 返回“新的 Context 指针”到 `trap.S`。
 - `trap.S` 执行 `mv sp, a0` 把栈指针切换到“目标 PCB 的内核栈上的 Context”，随后 POP 恢复寄存器，LOAD `mstatus/mepc`，`mret` 跳转到目标的 `mepc`。
 - 若目标为内核线程：从 `entry` 开始或从 `yield` 之后继续。
 - 若目标为用户进程：`mepc`=用户入口，用户态 `_start` 先 `sp←a0`，再进入 `main(argc, argv, envp)`。

- 定时器与协作让出
 - 本 PA 中主要通过协作式 `yield()` 实现切换（hello 每次循环，设备接口 `serial_write()/events_read()/fb_write()` 开头也 `yield()`），因此 PAL 在 I/O 时会让出 CPU。
 - 在 native 环境可收到 `EVENT_IRQ_TIMER`，示例中识别后直接返回当前上下文，不做强制抢占。
- 分页(VME)与地址空间（如启用）
 - `vme_init()` 建好内核页目录 `kas` 并 `satp-kas`；`protect(as)` 为进程创建顶级页表，拷贝内核映射；`map()` 建 `VA`→`PA` 映射。
 - 切换时机在 `__am_switch(Context *c)` 中 `set_satp(c->pdir)`，由 `__am_irq_handle`/恢复流程间接触发。
 - 作用：隔离用户/内核地址空间，保证 PAL 只能访问自己的代码/数据/栈；本 PA 可先不启用，逻辑保持一致。

结果：hello 频繁 `yield()`，PAL 在 I/O 前 `yield()`，两者在 `schedule()` 的选择下交替执行，达成分时多任务。

2. 字符串写保护机制的实现与验证

示例程序：

```
int main() {
    char *p = "abc";
    p[0] = 'A';
    return 0;
}
```

现象：运行触发“段错误”（SIGSEGV）。

分层分析：

- 程序与编译器
 - 字符串字面量存放在只读段 `.rodata`。`char *p = "abc"` 让 `p` 指向 `.rodata` 中的常量区；对其写入是未定义行为，现代系统通过内存保护使其必然失败。
- 链接器与可执行文件
 - 链接器将 `.text`、`.rodata`、`.data`/`.bss` 打包为 ELF Program Header 的 `PT_LOAD` 段，并设置权限标志：
 - `.text` → `PF_R|PF_X`
 - `.rodata` → `PF_R`
 - `.data` → `PF_R|PF_W`
- 运行时/加载器
 - 动态加载器或内核加载可执行文件时，根据 Program Header 用 `mmap` 将段映射到进程地址空间，并据此设置页权限（只读、可写、可执行）。
- 操作系统与硬件
 - 硬件分页(PTE)保存每页的权限位（x86: `R/W, U/S, XD`；RISC-V: `R/W/X/U`）。
 - 当执行 `p[0] = 'A'` 时，CPU 进行一次对 `.rodata` 页的写访问：

- 页存在，但 PTE 无写权限 → 触发页保护异常（x86 的 #PF，错误码表明保护违规；RISC-V 的 Store/AMO page fault，mcause=15）。
- OS 的页故障处理器判断该地址/权限非法，向进程发送 SIGSEGV（典型为 SEGV_ACCERR），默认动作终止进程 → “Segmentation fault”。

如何证明：

- 观察段权限
 - readelf/objdump：
 - readelf -S a.out 查看 .rodata
 - readelf -l a.out 查看 PT_LOAD 段的 R/E/W 标志
 - objdump -h a.out 查看节布局
- 观察进程映射
 - 运行前 ulimit -c unlimited 生成 core，或在 gdb 中运行
 - 在断点/崩溃处 cat /proc/\$(pidof a.out)/maps，可见 .text、.rodata 的权限分别为 r-xp、r--p
- 触发点与异常
 - gdb：
 - gcc -O0 -g segv.c -o segv
 - gdb ./segv
 - b main/r, ni 到 p[0]='A'
 - 观察崩溃信号：Program received signal SIGSEGV, Segmentation fault.
 - x86_64 可 disas 看到 movb 写指令；RISC-V 可见 sb aX,0(aY)
- 系统调用轨迹
 - strace ./segv 可见进程收到 SIGSEGV 并由默认处理器终止；dmesg/journalctl -k 可看到内核报告的页故障信息（含地址、错误码）。

保障机制总结：链接器标记只读段 → 加载器以只读方式映射 → OS 建立只读页表 → 硬件在写访问时强制触发页故障 → OS 送达 SIGSEGV。多层共同保证对字符串常量的写会失败，从而保护代码与常量数据的只读性。